Initial bipartition

Refinement needs a legal starting cut

The idea

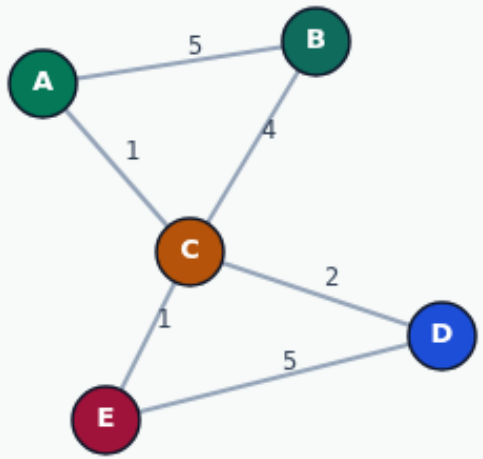
- Random shuffles then splits by half
- Greedy prefers keeping heavy edges internal when it can
- Grow expands a frontier from a seed until the part hits its size budget
- All three must report cutsize and balance so you can compare seeds fairly
- <!-- algorithm-walkthrough -->

Need a legal starting split

INITIAL BIPARTITION

Need a legal starting split

Step 1 / 5 · empty · module01-03-initial-bipartition



Explanation

Refinement (KL/FM) needs a bipartition seed. Common builders: random, greedy heaviest-edge, or grow-from-node BFS. All must end with exactly two sides.

- Random: shuffle + half/half
- Greedy: anchor heaviest edge
- Grow: BFS by heaviest neighbor

```
5 nodes → target ~2 vs 3 per side
```

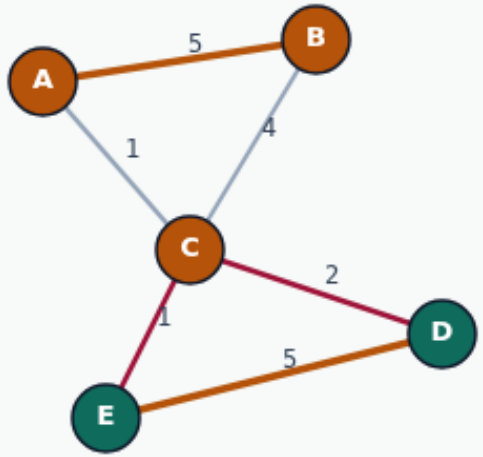
Need a legal starting split

Random seed=1: lucky ABC|DE

INITIAL BIPARTITION

Random seed=1: lucky ABC|DE

Step 2 / 5 · random-lucky · module01-03-initial-bipartition



Explanation

Random (seed=1) lands on ABC|DE with cutsize 3 — the golden communities by luck. Random seeds can also be terrible (seed=4 → cut 13).

- Reproducible with fixed RNG seed
- Lucky draw matches golden
- Unlucky seeds need refinement

```
method: random seed=1
parts: ABC|DE
cutsize: 3
```

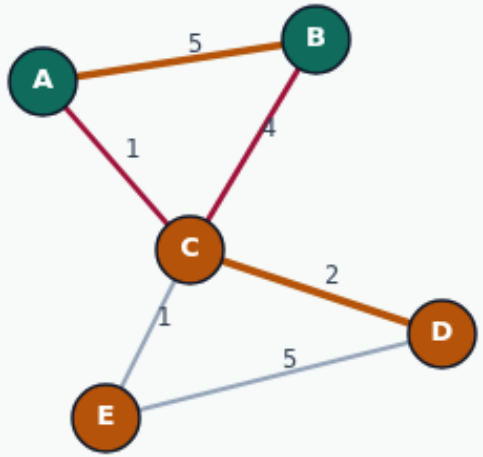
Random seed=1: lucky ABC|DE

Greedy initial: AB|CDE cut 5

INITIAL BIPARTITION

Greedy initial: AB|CDE cut 5

Step 3 / 5 · greedy · module01-03-initial-bipartition



Explanation

Greedy anchors heaviest edge A–B on side 0, then places remaining nodes to minimize added cut. Result AB|CDE has cutsize 5 — legal but worse than grow.

- Starts with A–B @ weight 5
- Fills side 1 for balance
- Deterministic, not always optimal

```
method: greedy
parts: AB|CDE
cutsize: 5
```

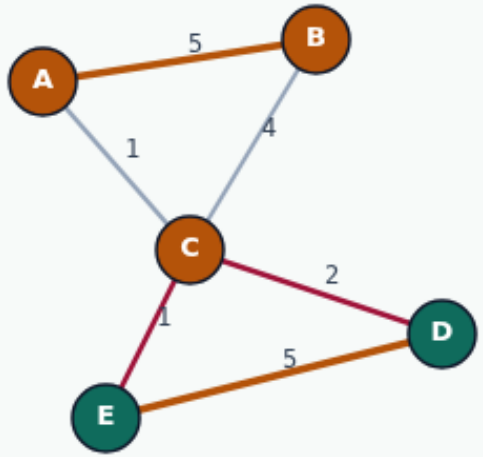
Greedy initial: AB|CDE cut 5

Grow from D: ABC|DE cut 3

INITIAL BIPARTITION

Grow from D: ABC|DE cut 3

Step 4 / 5 · grow-d · module01-03-initial-bipartition



Explanation

Grow from D pulls E first (w=5), then A and B via heavy edges. Side 0 = {D,E}, side 1 = {A,B,C} — cutsize 3, matching golden.

- BFS by heaviest uncut neighbor
- Grow from E is symmetric
- Reference starter in the lab

```
method: grow seed D
parts: ABC|DE
cutsize: 3
```

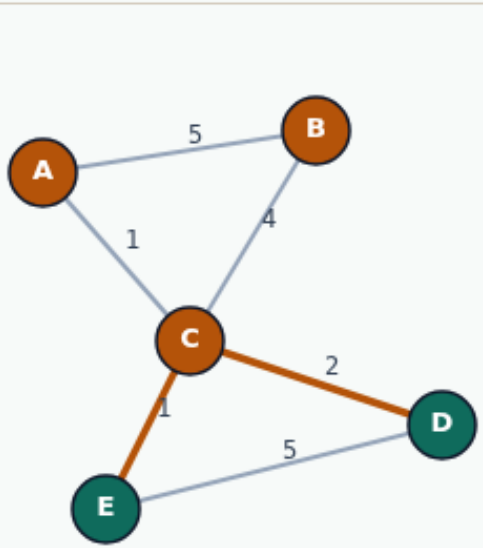
Grow from D: ABC|DE cut 3

Seed quality sets the ceiling

INITIAL BIPARTITION

Seed quality sets the ceiling

Step 5 / 5 · takeaway · module01-03-initial-bipartition



Explanation

All three methods produce legal bipartitions, but cutsize ranges 3–13 on the same graph. Better seeds mean less work for KL/FM downstream.

- Random: luck-dependent
- Greedy: cut 5 on this instance
- Grow D: cut 3 — lab reference

```
Always exactly 2 labels
Grow D beats greedy here
```

Seed quality sets the ceiling

Browser lab track

- In the browser lab track, open the **initial-bipartition** lab from the tools shelf
- Load the starter graph, run the algorithm once
- Work the challenges that lock the goldens

Implement track

- In the implement track, open this module’s examples and the course `common/` solvers
- Parse the tiny graph, run the algorithm with a deterministic seed
- Match the browser goldens before you claim the checklist

Pitfalls

- Common traps
- For multilevel flows, verify coarsening before you blame the refiner

Your turn

- Complete the checklist for at least one track, preferably both
- Implement until your metrics match the starter goldens
- When you’re ready, take the short quiz, then continue to the next module